Laporan: Akurasi Ekstraksi dan Performa Scraper Berita

- **Tanggal:** 2 November 2026
- **Lingkup:** `scraper/` (Bun + Chrome headless) dan `go-scraper/` (Go tanpa browser)
- **Rincian:** ADR-0003 · ADR-0004 (ditolak) · ADR-0005 · benchmark akurasi · benchmark driver

Ringkasan

- **Akurasi kelima implementasi sama: recall 100%, precision 100%** pada 24 artikel dari 8 sumber. Judul, tanggal terbit, penulis, dan gambar di dalam isi juga sama di semua implementasi.
- **Aturan ekstraksi dan pembersihnya disamakan.**
  - Aturan ekstraksi dan pembersih 11 aturan dari sistem produksi dipakai bersama oleh scraper Bun, Go (rules), dan Go (trafilatura).
  - Pada 60 artikel RSS terbaru, ke-11 pola sampah itu tersisa 0.
- **Yang membedakan adalah performa:**
  - **Go tanpa browser:** ±100–120 MB, dibanding ±1,6–2,0 GB untuk Chrome. Waktu per artikel paling pendek.
  - **agent-browser:** paling lambat, ±26 dtk per artikel.
  - **Biaya pemrosesan Go:** di luar waktu jaringan, ekstraksi Go (rules) ±8 ms per halaman dan Go (trafilatura) ±15 ms per halaman.
- **Ekstraksi dengan AI agent (Hermes) ditolak** (bagian 4).

1. Akurasi

Metode

- Hasil scraper dibandingkan dengan **halaman aslinya**, bukan dengan hasil scraper lain.
- Pembanding diambil dari HTML lewat request HTTP biasa, memakai selector yang dipilih manual per situs.
- Teks pembanding ikut dibersihkan dengan 11 aturan pembersih, jadi pola itu dihitung sebagai sampah.
- Sampelnya 24 artikel, 3 terbaru per sumber, dengan URL yang sama untuk semua implementasi dan semua putaran.
- Definisi:
  - **Recall:** bagian isi berita di halaman yang ikut terambil.
  - **Precision:** bagian teks hasil scraper yang benar-benar isi berita; sisanya iklan, footer, atau tautan.

Implementasi yang diukur:

Implementasi	Cara membuka halaman	Ekstraksi
Bun + browser (agent-browser, CDP Bun, Playwright)	Chrome headless	`extract-article.js`, dijalankan di halaman
Go (rules)	HTTP biasa, tanpa browser	Port 1:1 `extract-article.js` dan `scrape-article.js`
Go (trafilatura)	HTTP biasa, tanpa browser	go-trafilatura v1.12.2 memilih wadah isi; teks, metadata, gambar, dan pembersihan memakai aturan yang sama dengan Go (rules)

Ketiganya memakai aturan pagination yang sama. Tidak ada yang memeriksa robots.txt.

Hasil (24 artikel, semuanya berhasil dibaca ketiga implementasi)

	Bun + browser	Go (rules)	Go (trafilatura)
Recall	100%	100%	100%
Precision	100%	100%	100%
Paragraf hilang	0	0	0
Paragraf bukan berita	0	0	0
Judul	24/24	24/24	24/24
Tanggal terbit	18/18	18/18	18/18
Penulis	12/12	12/12	12/12
Gambar di dalam isi	28/31	28/31	28/31

- **Kolom "Bun + browser"** diukur dengan CDP Bun. Pada putaran performa, isi dan judul dari agent-browser, Playwright, Go (rules), dan Go (trafilatura) identik dengan CDP Bun di setiap artikel yang berhasil dibuka keduanya (22/22 sampai 23/23).
- **Tanggal dan penulis** hanya dinilai di situs yang menampilkannya di halaman (6 situs untuk tanggal, 4 situs untuk penulis).
- **Gambar di dalam isi:** 3 gambar yang tidak terambil berada di viva dan sama di semua implementasi.

Pembersih teks: 11 aturan dari data sistem produksi

Aturan pembersihnya didata tim produksi dan dipasang sama persis di bagian "Pembersihan teks" `extract-article.js` serta port Go-nya, `go-scraper/internal/infrastructure/extract/clean`. Empat belas contoh uji menghasilkan keluaran yang sama di JS dan Go.

No	Pola	Aturan
1	Kode redaksi `(lil/sfr)`	Hapus di ujung paragraf
2	`[Gambas:Video CNBC]`	Hapus di mana pun
3	Spasi sebelum tanda baca `kata .`	Rapikan menjadi `kata.`
4	Dateline `Jakarta, CNN/CNBC Indonesia -`	Hapus di awal teks
5	Spasi di dalam kurung `( x )`	Rapikan menjadi `(x)`
6, 9	Disclaimer TradingView dan promo Telegram sebelumnya	Potong dari `Disclaimer The information…` sampai akhir
7	"Baca juga" / "Lihat Juga" + judul	Hapus paragrafnya; jika menempel, hapus dari penanda sampai akhir paragraf
8	Promo video Investinglive	Hapus di ujung teks
10	`Magazine: …` Cointelegraph	Hapus di ujung teks
11	Kredit `Foto: …`	Hapus paragraf kredit pendek dan `(Foto: …)` di dalam kalimat

Pada 60 artikel RSS terbaru (20 per sumber dari cnnindonesia, detik, dan cnbcindonesia), pola 1, 2, 3, 4, 5, 7, dan 11 tersisa **0** di Go (rules) dan Go (trafilatura). Pola 6, 8, 9, dan 10 diuji dengan contoh dari tabel produksi saja, karena TradingView, Investinglive, dan Cointelegraph tidak ada di `config.json`.

Di sampel yang sama ada satu artikel cnnindonesia (daftar idol K-pop) tanpa wadah `#article-content-body`. Isinya kosong di Bun + browser dan Go (rules), tetapi terbaca di Go (trafilatura), karena go-trafilatura tetap menemukan wadahnya. Artikel seperti ini tertahan oleh `minWords` di scraper Bun dan tidak dikirim.

Efek tanpa browser

CDP Bun dan Go (rules) dijalankan berurutan pada ke-24 artikel, lalu hasilnya dibandingkan per artikel.

Aspek	Hasil
Teks isi	**Identik persis 24/24**: jumlah kata, jumlah paragraf, dan setiap karakter sama
Judul, tanggal terbit, penulis, deskripsi, tag	Identik 24/24
Daftar gambar	Sama di 20/24; berbeda di 4 artikel (rincian di bawah)

Artikel	Perbedaan	Penyebab
cnnindonesia (1 artikel)	Browser mengambil 9 gambar dari artikel lain ("Barcelona vs Deportivo")	Blok rekomendasi yang dimuat JavaScript; tidak selalu muncul
liputan6 (2 artikel)	Go mengambil thumbnail berita terkait (200×113) yang tidak ada di browser	Dugaan: di browser, JavaScript lazy-load mengganti atau menghapus elemen itu
detik (1 artikel)	Go mengambil placeholder lazy-load `loading.gif`	Dugaan: di browser, JavaScript mengganti placeholder dengan gambar asli

Batasan yang tidak muncul di 8 sumber ini, tetapi berlaku untuk sumber lain:

- isi atau link pagination yang baru muncul lewat JavaScript tidak terbaca tanpa browser;
- elemen yang disembunyikan lewat CSS tetap terbaca.

Aturan ekstraksi yang dipakai (`extract-article.js`)

1. **Daftar selector umum** situs berita, yang hasilnya diperiksa dengan **skor kepadatan paragraf** (mirip Readability). Situs dengan struktur yang belum dikenal tetap tertangani lewat skor ini.
2. **Filter elemen bukan berita** (iklan, "baca juga", share, penulis/tanggal, keterangan foto) dan **filter baris label** (`ADVERTISEMENT` , `Pewarta:/Editor:` , ajakan follow, label `Halaman:` , placeholder video, inisial redaksi).
3. Isi yang tidak dibungkus `<p>` (sindonews) diambil per baris.
4. **Pembersih teks** dengan 11 aturan di atas.

Setiap perubahan aturan diterapkan juga di port Go, lalu diukur ulang dengan `bun run bench:accuracy --urls <hasil sebelumnya>` .

2. Performa

Waktu per artikel dan memori (21 artikel yang berhasil dibuka kelima implementasi, tanpa pembukaan pertama)

Implementasi	Median / artikel	p95	Cold start	Peak memory	Berhasil
agent-browser (default saat ini)	26,2 dtk	54,4 dtk	20,2 dtk	1,8 GB (Chrome)	22/24
CDP Bun	6,6 dtk	22,8 dtk	1,9 dtk	2,0 GB (Chrome)	23/24
Playwright	6,2 dtk	32,3 dtk	14,7 dtk	1,6 GB (Chrome)	24/24
Go (rules)	4,7 dtk	13,2 dtk	1,6 dtk	**97 MB** (proses Go)	24/24
Go (trafilatura)	2,7 dtk	8,7 dtk	1,0 dtk	117 MB (proses Go)	24/24

Median rasio waktu per artikel terhadap Go (rules), pada URL yang sama: agent-browser 7,0×, CDP Bun 2,3×, Playwright 1,7×, Go (trafilatura) 0,86×.

Waktu ekstraksi tanpa jaringan

HTML ke-24 artikel diunduh sekali, lalu setiap ekstraktor Go dijalankan 5 kali per halaman. Hasilnya konsisten di dua putaran.

Ekstraktor	Median / halaman	p95	Total 24 halaman
Go (rules)	7,7–8,0 ms	9,3–9,6 ms	±170 ms
Go (trafilatura)	15,1–15,7 ms	17,4–18,5 ms	±340 ms

Go (trafilatura) memproses ±2× lebih lama dari Go (rules), karena ia menjalankan go-trafilatura di atas aturan rules. Keduanya hanya milidetik, jauh di bawah waktu jaringan yang dalam hitungan detik.

Cara membaca angka performa

- **Waktu per artikel didominasi jaringan**, bukan ekstraksi. Selisih antar implementasi Go di tabel pertama berasal dari jaringan dan urutan pengukuran, bukan dari ekstraktornya.
- **Ada bias urutan.** Urutan implementasi digilir per URL, tetapi di 4 dari 5 rotasi Go (trafilatura) berjalan tepat setelah Go (rules), dan Playwright tepat setelah CDP Bun. Implementasi kedua mendapat cache CDN yang baru dipanaskan untuk URL yang sama, sehingga Go (trafilatura) terlihat lebih cepat dari Go (rules) walaupun pemrosesannya 2× lebih lama.
- **Jaringan tidak stabil saat pengukuran.** Ada timeout di okezone untuk agent-browser dan CDP Bun. Di putaran lain saat jaringan normal, median CDP Bun 0,6 dtk dan Playwright 0,3 dtk (benchmark driver).
- **Yang konsisten di setiap putaran:** agent-browser selalu paling lambat; Go selalu lebih cepat dan jauh lebih hemat memori dari driver browser.
- **Cakupan memori tidak sama.** Untuk Go diukur seluruh prosesnya, untuk browser hanya pohon proses Chrome (proses Bun tidak ikut dihitung).

Penyebab selisih kecepatan

- `agent-browser open` menunggu event `load`, yaitu sampai semua iklan selesai dimuat; di situs penuh iklan waktunya bisa ±20 dtk.
- CDP Bun dan Playwright berhenti saat `DOMContentLoaded`, ketika isi berita sudah ada di halaman, tetapi tetap memuat JavaScript, gambar, dan iklan.
- Go hanya mengunduh HTML-nya.

3. Perbandingan implementasi

	agent-browser	Playwright	CDP Bun	Go (rules)	Go (trafilatura)
Akurasi (24 artikel)	100% / 100%	100% / 100%	100% / 100%	100% / 100%	100% / 100%
Waktu per artikel	Paling lambat (±26 dtk)	Cepat	Cepat	Tercepat	Tercepat
Waktu ekstraksi tanpa jaringan	-	-	-	±8 ms	±15 ms
Memori	±1,8 GB	±1,6 GB	±2,0 GB	±0,1 GB	±0,1 GB
Butuh Chrome di server	Ya	Ya	Ya	Tidak	Tidak
Isi yang dirender JavaScript	Terbaca	Terbaca	Terbaca	Tidak terbaca	Tidak terbaca
Dependency	Binary npm global (butuh Node.js/npm di server)	`playwright-core`	Tidak ada	`x/net/html`, `cascadia`	Ditambah go-trafilatura
Pemulihan crash Chrome	Belum ada	Otomatis	Otomatis	Tidak relevan	Tidak relevan

4. Evaluasi AI agent (Hermes Agent): ditolak

Dikaji apakah membaca isi artikel lewat AI agent lebih andal daripada ekstraktor deterministik. Agent yang diuji adalah Hermes Agent dengan Claude Sonnet 5.5: membuka halaman dengan browser, lalu menyalin isi berita sesuai prompt. Uji cobanya 1 artikel detik dan 1 artikel liputan6 per backend, dinilai dengan alat akurasi yang sama.

Cara	detik	liputan6	Waktu / artikel	Biaya / artikel
Scraper deterministik (CDP Bun)	100 / 100	100 / 100	0,2–2,6 dtk	$0
Hermes + Browser Use (bawaan Hermes)	100 / 100	100 / 100	54–55 dtk	$0,043–0,048
Hermes + agent-browser 0.26.0	gagal (timeout)	100 / 100	40 dtk	$0,051

Recall / precision dalam persen.

Ditolak (ADR-0004), karena:

- **Akurasi tidak bertambah.** Ekstraktor deterministik sudah 100% / 100% pada 8 sumber.
- **Selector cukup ditentukan saat setup** dan diverifikasi dengan benchmark akurasi; tidak perlu AI.
- **Mahal dan lambat:** ±$45–50 per 1.000 artikel (perkiraan) dan ±25–250× lebih lambat, tanpa imbalan akurasi.
- **Risiko keamanan:** backend yang andal (Browser Use) butuh akses shell untuk agent sambil membaca halaman tak tepercaya.
- **Tidak deterministik:** risiko parafrase atau halusinasi teks.
- **Izin penerbit:** pemrosesan isi berita oleh AI dilarang sebagian penerbit (mis. antaranews tanpa izin tertulis).

5. Usulan

- **Untuk 8 sumber ini, browser tidak dibutuhkan.**
    - HTML server sudah memuat isi lengkap.

    - Hasil tanpa browser sama dengan hasil browser.
    - Memori turun dari ±1,6–2,0 GB ke ±0,1 GB.
- **Pendekatan sistem produksi Go (go-trafilatura plus penanganan sendiri) setara akurasinya** dengan aturan scraper Bun jika memakai aturan yang sama. Pemilihan antara Go (rules) dan Go (trafilatura) cukup didasarkan pada:
    - **biaya pemrosesan:** ±8 vs ±15 ms per halaman;
    - **perawatan:** Go (rules) berarti satu set aturan di dua bahasa; Go (trafilatura) menambah dependency go-trafilatura;
    - **ketahanan terhadap struktur halaman yang belum dikenal:** go-trafilatura berhasil membaca 1 artikel yang gagal dibaca rules.
- **Browser tetap disiapkan sebagai jalur cadangan** untuk sumber yang isinya hanya muncul lewat JavaScript. Jika jalur browser dipakai, Playwright atau CDP Bun dipakai, bukan agent-browser.
- Rincian dan pertanyaan terbuka ada di ADR-0005.

6. Risiko dan batasan

- **Izin penerbit:** footer antaranews melarang pengindeksan otomatis untuk AI tanpa izin tertulis, dan robots.txt tempo memasang `ai-train=no`. Pemakaian produksi perlu dipastikan izinnya.
- **Aturan di beberapa tempat:** aturan ekstraksi dan pembersih ada di `extract-article.js`, di port Go, dan dipakai referensi benchmark. Setiap perubahan harus diterapkan di semua tempat dan diukur ulang.
- **Sumber terblokir:** tempo, republika, dan tribunnews memblokir Chrome headless dan tidak diukur.
- **Sampel kecil:** 24 artikel untuk akurasi dan 60 artikel untuk pola sampah, dari satu hari.

Sumber

Repo: github.com/adehikmatfr/news-scrap

Dokumen / kode	Isi
ADR-0003: driver browser	Pilihan driver lewat `BROWSER_DRIVER`, spike Windows/Linux, sandbox
ADR-0004: Hermes Agent (ditolak)	Data spike Hermes dan alasan penolakan
ADR-0005: komparasi scraper Go tanpa browser	Port Go, pembersih 11 aturan, hasil, usulan, dan pertanyaan terbuka
Benchmark akurasi ekstraksi	Data per artikel dan per sumber, sebelum dan sesudah perbaikan ekstraktor
Benchmark driver browser	Performa ketiga driver browser saat jaringan normal
`extract-article.js`	Aturan ekstraksi dan pembersih teks yang dipakai ketiga driver browser
`agent-browser-cli.js` · `cdp-driver.js` · `playwright-driver.js`	Implementasi ketiga driver browser
`go-scraper/`	Scraper Go tanpa browser (clean architecture)
`rules.go` · `clean.go` · `trafilatura.go` · `scrape_article.go`	Port Go dari `extract-article.js`, pembersih teks, adapter go-trafilatura, dan `scrape-article.js`
`compare-with-source.js`	Alat ukur akurasi (`bun run bench:accuracy`)
`compare-drivers.js`	Alat ukur performa (`bun run bench`)